

DIGIWIZARD

A MODULAR MULTI DEV TABLET

Submitted in partial fulfillment of the requirements for the award of
Bachelor of Engineering degree in Computer Science and Engineering

By

PRIYADHARSHAN P (Reg.No - 41111006)
MONISH ANAND.J (Reg.No – 41110815)



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING

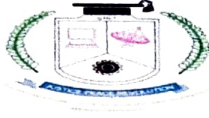
SATHYABAMA

INSTITUTE OF SCIENCE AND TECHNOLOGY
(DEEMED TO BE UNIVERSITY)

CATEGORY - 1 UNIVERSITY BY UGC

Accredited with Grade “A++” by NAAC | Approved by AICTE
JEPPIAAR NAGAR, RAJIV GANDHI SALAI, CHENNAI - 600119

APRIL - 2025



SATHYABAMA

INSTITUTE OF SCIENCE AND TECHNOLOGY
(DEEMED TO BE UNIVERSITY)

CATEGORY - 1 UNIVERSITY BY UGC

Accredited with Grade "A++" by NAAC | Approved by AICTE

www.sathyabama.ac.in

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

This is to certify that this Project Report is the bonafide work of **PRIYADHARSHAN P (41111006)** and **MONISH ANAND J (41110815)** who have done the Project work as a team who carried out the project entitled "**DIGIWIZARD - A MODULAR MULTI DEV TABLET**" under our supervision from November 2024 to April 2025.

T. Anandhi
1/4/25
Internal Guide

Ms. T. Anandhi, M.E., (Ph.D.),

Head of the Department

HEAD OF THE DEPARTMENT
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
Dr. L. LAKSHMANAN, M.E., Ph.D.
SATHYABAMA
INSTITUTE OF SCIENCE AND TECHNOLOGY
(DEEMED TO BE UNIVERSITY)
Jeppiaar Nagar, Rajiv Gandhi Salai,
Chennai - 600 119.

Submitted for Viva voce Examination held on 04-04-2025

Internal Examiner

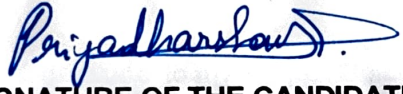
External Examiner

DECLARATION

I, **PRIYADHARSHAN P** (Reg.No - 41111006) hereby declare that the Project Report entitled "**DIGIWIZARD - A MODULAR MULTI DEV TABLET**" done by me under the guidance of **Ms. T. Anandhi., M.E., (Ph.D.)**, is submitted in partial fulfillment of the requirements for the award of Bachelor of Engineering degree in **Computer Science and Engineering**.

DATE: 04-04-2025

PLACE: Chennai



SIGNATURE OF THE CANDIDATE

ACKNOWLEDGEMENT

I am pleased to acknowledge my sincere thanks to **Board of Management of Sathyabama Institute of Science and Technology** for their kind encouragement in doing this project and for completing it successfully. I am grateful to them.

I convey my thanks to **Dr. T. Sasikala M.E., Ph. D, Dean**, School of Computing, and **Dr. L. Lakshmanan M.E., Ph.D.**, Head of the Department of Computer Science and Engineering for providing me necessary support and details at the right time during the progressive reviews.

I would like to express my sincere and deep sense of gratitude to my Project Guide **Ms.T.Anandhi M.E., (Ph.D.)**, for her valuable guidance, suggestions, and constant encouragement paved way for the successful completion of my project work.

I wish to express my thanks to all Teaching and Non-teaching staff members of the **Department of Computer Science and Engineering** who were helpful in many ways for the completion of the project.

ABSTRACT

This project focuses on developing a Raspberry Pi-based tablet with Infrared (IR) and RFID/NFC emulation, enabling it to function as a universal remote control and a secure access management system. Designed as a cost-effective and customizable solution, it supports applications in smart home automation, IoT prototyping, and education. By integrating multiple wireless communication technologies, the device enhances convenience, security, and efficiency. The IR module allows seamless control of home appliances like televisions and air conditioners, while the NFC/RFID module enables secure contactless authentication for access control, digital payments, and identity verification. The system's versatility lies in its ability to combine remote control, access management, and data transfer within a single compact device, making it suitable for both personal and industrial use. Its open-source architecture allows customization, while optimized power consumption ensures long-term efficiency. With secure data handling and modular expandability, this project advances embedded systems, automation, and IoT solutions, offering a scalable and adaptable platform for future innovations in smart technology.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	v
	LIST OF FIGURES	vii
	LIST OF TABLES	viii
1	INTRODUCTION	1
2	LITERATURE SURVEY	
	2.1 Inferences from the Literature Survey	10
	2.2 Open Problems in Existing System	11
3	REQUIREMENTS ANALYSIS	
	3.1 Feasibility Studies/Risk Analysis of the Project	12
	3.2 Hardware and Software Requirements	15
	3.2.1 Hardware Requirements	15
	3.2.2 Software Requirements	18
4	DESCRIPTION OF PROPOSED SYSTEM	
	4.1 Selected Methodology	20
	4.2 Architecture of Proposed System	22
	4.3 Description of Modules	25
	4.4 Implementation	28
	4.5 Financial report on estimated costing	30
5	RESULTS AND DISCUSSION	
	5.1 Analysis and Discussion of Results	32
	5.2 Future Enhancements	33
6	CONCLUSION	34
	REFERENCES	35
	APPENDIX	37
	A. SOURCE CODE	37
	B. SCREENSHOTS	55
	C. RESEARCH PAPER	59

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
3.2.1	Raspberry Pi 5	16
3.2.2	Touchscreen Display for Raspberry Pi 5	16
3.2.3	IR Sensor	16
3.2.4	RFID NFC Module (PN532)	17
3.2.5	Power Supply	17
3.2.6	Additional Components	17
3.2.7	OS Details	18
3.2.8	Development Environment	19
4.2.1	Architecture Diagram	24
4.2.2	Data flow Diagram	24
4.2.3	Class Diagram	25
5.1	RFID/NFC Authentication Process Flow	32
B.1	Post Execution	55
B.2	Initial Script Screen	55
B.3	Changing All protocols	56
B.4	Terminal Runing the Test	56
B.5	Application List view	57
B.6	Task Switcher	57
B.7	Home Screen	57
B.8	Customized KDE Control Centre	58
B.9	Custom GUI Settings	58
B.10	Lock Screen	58

LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO.
4.5.1	Hardware Components Expenses	30
4.5.2	Software Components Expenses	30
4.5.3	Overhead Expenses	31
5.1.1	Response Time of IR Sensor	33

CHAPTER 1

INTRODUCTION

In the rapidly evolving technological landscape, the need for multi-functional and adaptive computing devices has become increasingly prominent. DIGIWIZARD is designed as a modular multi-device tablet powered by the Raspberry Pi, integrating Infrared (IR) and RFID/NFC emulation capabilities. This device serves as an advanced, portable solution for various applications, including home automation, secure access control, and interactive learning.

Traditional tablets primarily focus on general computing tasks such as web browsing, media playback, and productivity applications. However, they often lack the ability to interact with external electronic devices seamlessly. DIGIWIZARD bridges this gap by offering a multi-protocol communication platform, allowing users to control appliances, authenticate via RFID/NFC, and customize functionalities based on their specific needs.

The core concept of this project is to provide a highly flexible and expandable device that enhances user experience through intelligent integration of hardware and software. Using IR transmission, DIGIWIZARD can replicate signals from remote controls to operate TVs, air conditioners, and other IR-based electronics. Meanwhile, RFID/NFC emulation enables it to read and emulate access cards, making it a useful tool for authentication and security purposes.

Furthermore, DIGIWIZARD is built on the Raspberry Pi platform, which ensures affordability, easy customization, and compatibility with various software frameworks. The Raspberry Pi's GPIO (General Purpose Input/Output) pins allow for seamless integration of additional sensors and modules, making it a powerful multi-functional hub for developers, students, and professionals.

Key Features of DIGIWIZARD

1. **Touchscreen Interface:** The device features a responsive touchscreen, offering an intuitive and user-friendly interface for easy navigation and control.
2. **Infrared (IR) Emulation:** Enables the device to function as a universal remote, capable of controlling TVs, air conditioners, and other IR-enabled appliances.
3. **RFID/NFC Integration:** Supports RFID/NFC tag reading and emulation for authentication, access control, and secure transactions.
4. **Modular Expandability:** Designed to support additional hardware modules, making it adaptable for various applications, including IoT and automation.
5. **Wireless Connectivity:** Equipped with Wi-Fi and Bluetooth for seamless communication with smart home systems, IoT devices, and cloud platforms.
6. **Portable and Power-Efficient:** Optimized for battery-powered operation, ensuring portability and prolonged usage.

Significance of the Project

DIGIWIZARD aims to revolutionize how users interact with their environment by consolidating multiple functionalities into a single, compact device. It has vast potential across multiple domains, including:

- **Home Automation:** Control smart appliances effortlessly without requiring multiple remote controls.
- **Security and Access Control:** Use RFID/NFC authentication for secure access to restricted areas.
- **Education and Learning:** Act as a learning tool for students interested in embedded systems and wireless communication.
- **Healthcare and Retail:** Assist in contactless transactions, patient authentication, and inventory management.

CHAPTER 2

LITERATURE SURVEY

The increasing reliance on contactless communication technologies such as Near Field Communication (NFC), Radio Frequency Identification (RFID), and Infrared (IR) has paved the way for innovative applications in security, authentication, and smart systems. These technologies offer seamless connectivity and enhanced convenience, making them widely adopted in industries such as finance, healthcare, transportation, and access control.

Despite their benefits, NFC, RFID, and IR technologies also introduce security risks, including unauthorized access, data interception, and hardware-based vulnerabilities. Researchers have explored various approaches to improve security, efficiency, and functionality, leading to advancements in encryption methods, device authentication, and secure data transmission.

This literature survey presents an overview of existing research in the field, analyzing different studies that focus on NFC-based authentication, RFID security challenges, IR-based communication models, and the integration of these technologies in modern security systems. The survey highlights the strengths and limitations of previous works, providing a foundation for understanding current trends and identifying potential areas for improvement.

Review on Existing System

- 1. A Survey on Near Field Communication (NFC) Technology.,Vedat Coskun · Busra Ozdenizci · Kerem Ok (2013). A Survey on Near Field Communication (NFC) Technology. Wireless personal communications, 71(3), 2259-2294.**

Near Field Communication (NFC) as a promising short range wireless communication technology facilitates mobile phone usage of billions of people throughout the world that offers diverse services ranging from payment and loyalty applications to access keys for offices and houses. Eventually NFC

technology integrates all such services into one single mobile phone. NFC technology has emerged lately, and consequently not much academic source is available yet. On the contrary, due to its promising business case options, there will be an increasing amount of work to be studied in the very close future. This paper presents the concept of NFC technology in a holistic approach with different perspectives, including communication essentials with standards, ecosystem and business issues, applications, and security issues. Open research areas and further recommended studies in terms of academic and business point of view are also explored and discussed at the end of each major subject's subsection. This comprehensive survey will be a valuable guide for researchers and academicians as well as for business world interested in NFC technology.

2. Navigating The Flipper Zero, A Comprehensive Tool for Cybersecurity Professionals
International Journal of Research Publication and Reviews
Journal homepage: www.ijrpr.com ISSN 2582-7421.

The Flipper Zero represents a paradigm shift in the realm of cybersecurity and penetration testing tools. Developed as an all-in-one hacking device, it integrates various functionalities including radio protocols analysis, RFID emulation, infrared signals manipulation, and more, all within a compact and user-friendly form factor. This paper explores the capabilities of the Flipper Zero, its potential applications in security assessments, and its implications for the future of digital security. Through a combination of hardware and software analysis, we delve into the intricacies of its design, examining its strengths and limitations. Furthermore, we discuss ethical considerations surrounding the use of such devices and propose safeguards to mitigate potential misuse. By shedding light on the Flipper Zero, this paper aims to contribute to the ongoing discourse on cybersecurity tools and empower professionals in safeguarding digital infrastructure.

**3. Hardware Hacking: An Approach to Evaluate the Importance of Hardware Hacking and its Associated Risks.,Mr.Hitendra Swami NLIU,Bhopal,M.P.,
Dr.SatyaPrakash NLIU,Bhopal,M.P. Dr.Shrish Kumar Tiwari
CUG,Gandhinagar,Gujrat.,International Journal of Novel Research and**

Hacking is not just limited to the software aspect of the cyberspace; hardware aspect of hacking has the same relevance and importance. as we saw a lot of hardware hacking attacks in past, like Stuxnet attack we should not neglect the presence of these cyber attacks which involve hardware hacking as they include a wide variety of targeted system such as, computer systems, IOTs servers, networks, automobiles, as well as medical implants too. In this research paper Researchers tried to explore the hardware hacking tools and its associated risk and its avoided side of cyber security.

4. Implementation of Host Card Emulation Mode Over Android Smartphone as Alternative ISO 14443A for Arduino NFC Shield is authored by Rochman Saifullah Basyari, Surya Michrandi Nasution, and Burhanuddin Dirgantara. They are affiliated with the Electrical Engineering Faculty at Telkom University in Bandung, Indonesia. This work was presented at the 2015 International Conference on Control, Electronics, Renewable Energy and Communications (ICCSC).

The development of today's technology is increasing, making it more sophisticated and easier for all human activities. Some activities, such as accessing or locking room doors, are usually done manually or with a smartcard, requiring us to carry various locks (house, garage, office, car) or a smartcard. Often, we forget to bring or keep these essential items because we are more accustomed to carrying smartphones everywhere. To address this issue, there is an idea to replace regular keys or smartcards with smartphones. This can be achieved by implementing the Near Field Communication (NFC) technology module in smartphones. By integrating NFC, the need to carry multiple items can be eliminated, and it will help those who frequently forget keys or smartcards. If Host-based Card Emulation (HCE) mode, introduced in Android OS 4.4 (KitKat), is implemented, NFC-enabled smartphones can function as smartcards. HCE mode allows smartphones with NFC modules to act as smartcards, enabling users to use one device for various activities. To emulate smartcards and exchange data between NFC-enabled smartphones and NFC

readers, components like the Arduino UNO and NFC shield PN532, using the smartcard standard ISO14443A, can be utilized.

5. Face Recognition System Based on Raspberry Pi Platform., Nafis Mustakim.,Noushad Hossain., Mohammad Mustafizur Rahman., Nadimul Islam., Zayed Hossain Sayem., Md. Asaduz Zaman Mamun.,Department of Electrical

and Electronic Engineering, Shahjalal University of Science and Technology,Science and Technology, Sylhet-3114, Bangladesh.,1st International Conference on Advances in Science, Engineering and Robotics Technology 2019 (ICASERT 2019)

Human face recognition plays an important role in video surveillance, human-computer interface, personalizing different applications. In this paper, we present an approach to detect and identify a human face from the real-time video that tracks a face and compares it with stored data of known individuals. Our approach recognizes an individual within a fraction of a second which completely ignores any background effect. It also shows additional information about that individual. Besides, this method also works on different lighting conditions which make it suitable to execute its purpose in a wide variety of environment without encountering any significant error. We have implemented our work using a Raspberry Pi. And this approach utilizes the benefits offered by different python libraries like OpenCV and numPy. SQLite database management system is used as the backbone of its memory. Through its faster and authentic performance, this system is worthy enough to be integrated with any applications to boost up automation and user-friendliness.

6. Design and Implementation of Raspberry Pi Based Cell Phone is authored by Ali Mohamed Abd-Elrahim, Ahmad Almustafa Abu-Algasim Mohammad, Al-Hussein Ali Hassan, Ali Abu-Assal, Al-Imam Muhammad Al-Imam, and Muhi-Aldeen Mohammad Muhi-Aldeen. Abd-Elrahim, Abu-Algasim Mohammad, Ali Hassan, Al-Imam, and Muhi-Aldeen are affiliated with the Department of Electronic Systems Engineering at the University of Garden City in Khartoum, Sudan. Ali Abu-Assal is affiliated with the Department of Electronic Engineering at the University of York in York,

UK. This work was presented at the 2020 International Conference on Computer, Control, Electrical, and Electronics Engineering (ICCEEE).

Smartphone OS are typically either closed to modification like iOS or resource-intensive with many deficiencies like Android, including issues with security and high cost. The design objectives for a new smartphone include achieving flexible software, adaptive hardware, extended battery life, enhanced capacity management, improved security, reduced cost, added IoT functionalities, and customizable accessories such as screens, touch pens, keyboards, mice, and batteries. The designed smart cellular phone utilizes a Raspberry Pi, GSM module (Pi-talk shield), touch screen, memory card, and power supply. The Raspberry Pi handles data processing and storage, while the GSM module manages calls and text messages. The system operates on the NOOBs OS, with Python code used for programming due to its strength, simplicity, and flexibility. Internet connectivity is provided via the Raspberry Pi's Wi-Fi terminal, supporting various IoT applications. The Pi-talk shield interfaces with the Raspberry Pi through GPIO pins, and the display screen is connected via HDMI. The Pi camera is attached through its specific terminal. The software has been built and tested, showing reliable performance in voice calls, SMS, and internet browsing, though further improvements are needed.

- 7. Visible-to-Mid-Infrared In-Sensor Computing With a Reconfigurable Black Phosphorus Photodiode is authored by Linfeng Wu, Mengdie Shi, Zhiyuan Dai, Tao Ye, Jie Deng, Yu Yu, Ruowen Wang, Yonghao Bu, Tianyuan Cui, Jiajun Ma, Huiming Luo, Junwei Huang, Yujie Zhang, Jing Zhou, and Xiaoshuang Chen. This work was published in IEEE Electron Device Letters, Vol. 45, No. 7, July 2024.**

In-sensor computing addresses issues of communication latency and power consumption found in traditional machine vision systems that separate image acquisition from processing. Most sensors with internal computing capabilities operate within the visible or near-infrared spectrum; however, there is a need to expand into the mid-infrared range. We introduce a reconfigurable black phosphorus (BP) photodiode designed for visible-to-mid-infrared in-sensor convolutional processing. This device employs split-gate induced spatially differential doping to create a reconfigurable isotype homojunction in the BP

channel, which allows for a controllable bipolar responsivity that varies linearly with the gating voltage. Operating at room temperature, the device's convolutional preprocessing is consistent with simulation results, and its application in handwritten digit recognition shows high accuracy. This advancement contributes to the development of broadband in-sensor computing and enhances intelligent detection technologies.

- 8. A Performance Evaluation of Raspberry Pi Zero W Based Gateway Running MQTT Broker for IoT is authored by Diana Bezerra Correia Lima, Rubens Matheus Brasil da Silva Lima, Douglas de Farias Medeiros, Renata Imaculada Soares Pereira, Cleonilson Protasio de Souza, and Orlando Baiocchi. Lima, Silva Lima, Medeiros, and Souza This work was partially funded by the National Institute of Science and Technology of Micro and Nanoelectronics Systems (Proc. 573738/2008-4 INCT NAMITEC), Maranhão Research Foundation (FAPEMA, Grant: 01550/16), and PNPB CAPES-Brazil (Grant Procs. PNPB 2878/2011). Souza also acknowledges support from the Research Productivity Scholarship – CNPq (Proc. 311794/20165 DT). Additional financial support was provided by CNPq and the Fundação de Apoio à Pesquisa do Estado da Paraíba - Brazil (FAPESQ) under PRONEX.**

The Internet of Things (IoT) has seen widespread adoption in recent years across various applications, including weather monitoring, transportation, smart homes, smart cities, and smart farms. The IoT ecosystem is extensive, encompassing everything from sensors and hardware devices to cloud computing. A growing focus within this ecosystem is edge computing, with the edge-computing gateway (GTW) being a key component. These gateways serve as data-processing devices closer to the source and as bridges to the Internet. Essential features for GTWs include robustness and efficiency, and a popular solution is the use of low-cost Raspberry Pi credit card-sized computers. The Message Queue Telemetry Transport (MQTT) protocol is favoured for IoT due to its low-power capabilities. This paper evaluates the performance of the Raspberry Pi Zero W, a low-power member of the Raspberry Pi family, used as an IoT gateway running MQTT. The study assesses performance based on metrics such as processor temperature, CPU usage, and the rate of received

MQTT messages under varying Quality of Service (QoS) levels.

9. Patel, M., & Sharma, R. (2020). "NFC and RFID in Automation: Current Trends and Future Prospects," in Automation Journal, 19(6), 335-349.

The integration of Near Field Communication (NFC) and Radio Frequency Identification (RFID) technologies into automation systems has witnessed significant advancements in recent years. This paper explores the current trends in the deployment of NFC and RFID within automated environments, highlighting their applications in various industries including manufacturing, logistics, and smart infrastructure. Through detailed analysis, the paper discusses the technological innovations driving the adoption of NFC and RFID, as well as the key challenges and barriers to their widespread implementation. The future prospects section outlines potential directions for research and development, emphasizing the need for improved interoperability, security, and cost-effectiveness. This study aims to offer valuable insights for researchers, practitioners, and policymakers involved in the advancement of automation technologies.

10. Khan, A., & Gupta, V. (2019). "Integrated Smart Systems: Leveraging IR, RFID, and NFC Technologies," in Smart Systems and Technologies, 8(1), 89-103.

This paper presents a comprehensive study on the integration of Infrared (IR), Radio Frequency Identification (RFID), and Near Field Communication (NFC) technologies within smart systems. The authors, Khan and Gupta, delve into the synergies created by leveraging these technologies to enhance the functionality, efficiency, and intelligence of smart systems across various applications. The paper outlines the current state of these technologies, their individual strengths, and the benefits of their combined use in creating more responsive and interconnected smart environments. Through a series of case studies and practical implementations, the authors illustrate how integrated smart systems can be developed to address challenges in sectors such as healthcare, transportation, and home automation. The discussion also highlights the technical and logistical hurdles that must be overcome to achieve seamless integration and widespread adoption.

2.1. Inferences and Challenges in Existing System

The examination of current technologies and systems provides insights into the potential and challenges associated with Near Field Communication (NFC), cybersecurity tools, hardware hacking, Raspberry Pi applications, and IoT systems. Understanding these elements is vital for developing innovative solutions and addressing the limitations of existing technologies.

Inferences

1. Integration and Versatility:

- NFC Technology is becoming an integral part of mobile devices, facilitating services like mobile payments and access control. Its integration streamlines interactions in various sectors.
- Cybersecurity Tools like Flipper Zero showcase a trend toward multifunctional devices, enhancing professionals' ability to conduct comprehensive security assessments.
- IoT and Edge Computing emphasize efficient data processing, reducing latency and improving communication through protocols like MQTT.

2. User Experience and Accessibility:

- NFC and Smart Devices simplify user interactions by consolidating physical keys and cards, offering a more convenient user experience.
- Raspberry Pi enables customizable solutions for educational and professional projects, providing an accessible platform for innovation.

3. Business Opportunities:

- NFC in Mobile Payments presents significant business potential, enabling companies to offer innovative services and personalized marketing, boosting customer engagement and retention.

2.2 Open Problems in Existing System

1. Security Concerns:

- NFC Security faces vulnerabilities such as unauthorized access and data interception, necessitating robust encryption and authentication.
- Cybersecurity Tools require ethical guidelines to prevent misuse, ensuring responsible usage.
- Hardware Hacking poses risks, demanding enhanced security measures to protect critical systems.

2. Interoperability and Standardization:

- NFC and IoT Protocols face interoperability issues due to varying standards, highlighting the need for universal standards and consistent performance across platforms.

3. Technical Limitations:

- NFC Range Constraints and Raspberry Pi's processing limitations pose challenges in extending functionality and performance for complex applications.

CHAPTER 3

REQUIREMENT ANALYSIS

3.1 FEASIBILITY STUDIES / RISK ANALYSIS OF THE PROJECT

The DigiWizard project is designed to integrate Infrared (IR) and RFID/NFC emulation into a compact, tablet-like device powered by a Raspberry Pi. The primary objective of this device is to serve as a universal controller for smart homes, access control systems, and IoT-based applications, offering a seamless user experience while reducing dependency on multiple standalone devices. By combining IR remote control functionality and RFID/NFC-based authentication, the system provides a versatile and cost-effective solution for home automation, security, and wireless communication.

The feasibility of the DigiWizard project is assessed across multiple aspects, ensuring that the device is both practical and scalable for real-world applications.

3.1.1 Feasibility Analysis

Technical Feasibility

- The Raspberry Pi is a powerful single-board computer capable of handling IR signal transmission and RFID/NFC emulation.
- The system can utilize open-source software such as LIRC (Linux Infrared Remote Control) for IR communication and nfcpy (Python NFC library) for RFID/NFC functions.
- The device is modular, meaning additional features such as Wi-Fi connectivity, Bluetooth control, and gesture recognition can be added later.
- The system is designed to process real-time signals, allowing quick response times for user inputs.

Economic Feasibility

- The project is built using low-cost components, making it an affordable alternative to expensive smart controllers.
- Using open-source libraries eliminates the need for paid software, reducing overall costs.
- The device combines multiple functionalities (IR control, RFID/NFC authentication), reducing the need to purchase separate hardware, thus making it cost-effective.
- The system requires minimal maintenance, and firmware updates can be delivered through over-the-air (OTA) updates, reducing long-term operational costs.

Operational Feasibility

- The device features a user-friendly touchscreen interface, making it accessible to both technical and non-technical users.
- It can be customized to work with different types of IR-controlled appliances, making it versatile.
- The RFID/NFC authentication system provides secure access control, making it suitable for homes, offices, and industries.
- The system supports multiple user profiles, allowing different levels of access for different users.
- With a portable and lightweight design, DigiWizard can be easily used in various environments without requiring significant installation efforts.

Legal and Ethical Feasibility

- The device complies with wireless communication standards to avoid interference with existing smart home devices.
- Data privacy and encryption mechanisms ensure that RFID/NFC credentials remain protected from unauthorized access.

- The IR-based system does not interfere with regulated radio frequencies, making it legally acceptable for use in consumer electronics.
- The project adheres to open-source licensing policies, ensuring compliance with software development regulations.

3.1.2 Risk Analysis

Despite its promising feasibility, the DigiWizard project comes with certain risks and challenges that need to be addressed:

Hardware Limitations

- Raspberry Pi's processing power might limit real-time emulation of IR and NFC signals under high loads.
- Overheating issues may arise if multiple functionalities run simultaneously without proper cooling mechanisms.

Power Consumption

- Continuous use of IR transmission and RFID/NFC scanning may drain power quickly, requiring an efficient power management system.
- Battery life needs optimization for extended portability.

Security Concerns

- Unauthorized access to RFID/NFC credentials could lead to security vulnerabilities.
- Data encryption techniques such as AES (Advanced Encryption Standard) should be implemented to secure communication.

Connectivity Challenges

- Wireless communication modules (Wi-Fi, Bluetooth) may face latency issues in congested network environments.
- External interference could affect IR signal transmission in bright or obstructed environments.

User Adoption and Compatibility

- Users may find it difficult to transition from traditional remotes to a digital touch-based universal controller.
- Compatibility issues might arise with older IR-based appliances that use non-standard signal frequencies.

Addressing these challenges through hardware upgrades, optimized power management, enhanced security protocols, and user training will ensure the long-term success of the DigiWizard project.

3.2 HARDWARE AND SOFTWARE REQUIREMENTS

The DigiWizard project relies on a combination of hardware components and software frameworks to achieve its functionalities.

3.2.1 Hardware Requirements

The proposed system is built on a combination of **hardware and software components**, each carefully selected to ensure performance, efficiency, and ease of use.

Hardware Specifications

1. Raspberry Pi 5

- Quad-core CPU for efficient processing
- Built-in Wi-Fi and Bluetooth for seamless connectivity
- Multiple USB ports and HDMI support for external peripherals



Fig.3.2.1. Raspberry Pi 5

2. Touchscreen Display

- 7-inch display for user-friendly interaction
- Resolution: 800x480 or higher
- Compatible with Raspberry Pi display port for easy integration



Fig.3.2.2 Touchscreen Display for Raspberry Pi 5

3. IR Transmitter

- Infrared LED paired with appropriate resistors
- Connected to Raspberry Pi GPIO pins for signal transmission



Fig.3.2.3. IR Sensor

4. RFID/NFC Reader/Writer Module

- Module: PN532 or MFRC522 for efficient card/tag reading
- Uses I2C or SPI interface for reliable communication with Raspberry Pi

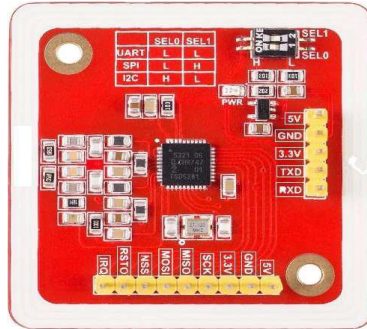


Fig.3.2.4. RFID NFC Module (PN532)

5. Power Supply

- 5V/3A adapter for stable power delivery
- Optional: Portable battery pack for mobile use



Fig.3.2.5. Power Supply

6. Additional Components

- HDMI cables, GPIO connectors, and jumper wires for assembly

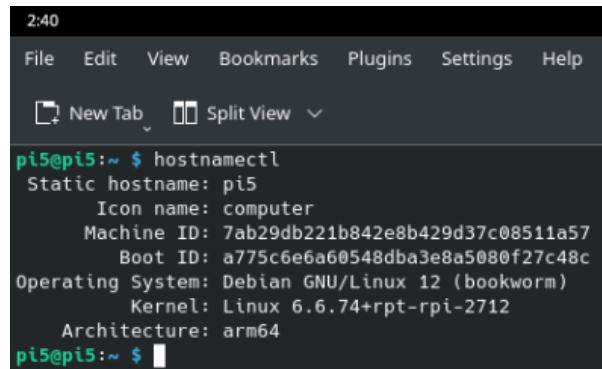


Fig.3.2.6. Additional component

3.2.2 Software Requirements

1. Operating System

- Raspberry Pi OS (default choice for compatibility)
- Alternatives: Ubuntu for additional flexibility



```
2:40
File Edit View Bookmarks Plugins Settings Help
New Tab Split View
pi5@pi5:~ $ hostnamectl
Static hostname: pi5
Icon name: computer
Machine ID: 7ab29db221b842e8b429d37c08511a57
Boot ID: a775c6e6a60548dba3e8a5080f27c48c
Operating System: Debian GNU/Linux 12 (bookworm)
Kernel: Linux 6.6.74+rpt-rpi-2712
Architecture: arm64
pi5@pi5:~ $
```

Fig.3.2.7. OS Details

2. Programming Languages

- **Python** (Primary language for development due to its extensive library support)
- **Bash** (Used for performance-critical functions where needed)
- **XML** (Configuration files).

3. Libraries and Tools

- **lirc**, **ir-keytable**, **gpiozero** for IR transmission
- **nfcpy**, **PN532**, **libnfc-tools-bin** for RFID/NFC handling
- **Tkinter** and **PyQT5** for designing the graphical user interface

4. Development Environment

- IDEs: **Geanny**, **Visual Studio Code** and **PyCharm** for coding
- **Git** for version control and project management
- **Terminal** for kernel level editing.

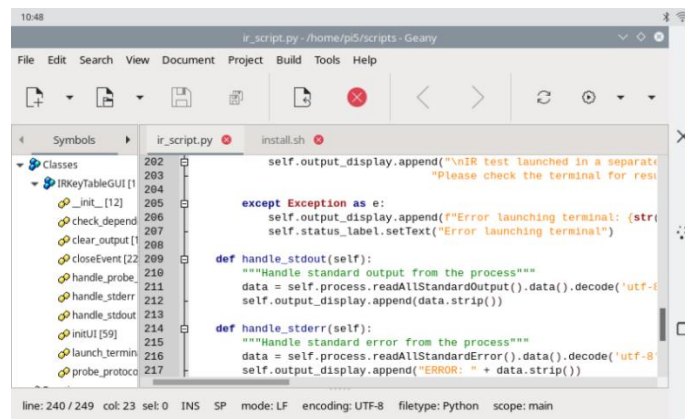


Fig.3.2.8. Development Environment

5. Security and Additional Software

- **Encryption tools** to secure wireless communications
- **Authentication mechanisms** to restrict unauthorized access
- Essential applications like a web browser and media player for extended usability

CHAPTER 4

DESCRIPTION OF PROPOSED SYSTEM

This proposed system is designed to provide seamless IR (Infrared) and RFID/NFC (Radio-Frequency Identification / Near-Field Communication) emulation through an interactive touchscreen interface. The system is built to enhance user experience by integrating multiple technologies, allowing efficient control over smart devices and secure communication with RFID/NFC-enabled systems.

This chapter details the methodologies, system architecture, functional modules, and implementation cost of DigiWizard. The proposed system follows an Agile Development Model with a Prototyping Approach to ensure iterative improvements and user feedback incorporation. The architecture comprises hardware components like a Raspberry Pi, IR transmitter, RFID/NFC module, and touchscreen display, along with software elements that enable smooth interaction and control.

The subsequent sections provide a structured breakdown of the system's methodologies, architectural design, module descriptions, and cost estimation for implementation.

4.1 SELECTED METHODOLOGY OR PROCESS MODEL

Agile Development Methodology

The Agile methodology is chosen for this project due to its flexibility and adaptability to changes during the development process. This approach is particularly suitable for DigiWizard, as it involves integrating multiple technologies and iterating on design and functionality.

1. Incremental and Iterative Development:

- Development is divided into small, manageable iterations that allow for continuous feedback and improvement.
- Each iteration focuses on implementing a specific set of features, enabling rapid prototyping and testing.

2. Collaboration and Communication:

- Emphasizes collaboration between team members, stakeholders, and end-users to ensure alignment with project goals and user needs.
- Regular meetings and updates facilitate open communication and transparency.

3. Adaptability:

- The Agile approach allows for flexibility in responding to changes in requirements, technology advancements, and user feedback.
- Encourages continuous evaluation and adjustment of project plans to ensure optimal outcomes.

4. User-Centric Design:

- Focuses on delivering a user-friendly interface and experience, with ongoing user involvement in the design and development process.
- Prioritizes features and functionalities that align with user needs and preferences.

Prototyping Methodology

Prototyping is used in conjunction with Agile development to create functional prototypes of DigiWizard. This method allows for early visualization of the product and helps in identifying potential issues and improvements.

1. Low-Fidelity Prototyping:

- Initial sketches and wireframes are created to outline the basic structure and functionality of the system.
- Provides a visual representation of the interface and user interactions.

2. High-Fidelity Prototyping:

- Advanced prototypes with detailed features and interactions are developed to simulate the final product.
- Allows for testing and validation of design choices and technical feasibility.

3. User Testing and Feedback:

- Prototypes are tested with users to gather feedback and insights on usability, functionality, and design.
- Iterative improvements are made based on user input to enhance the overall user experience.

4.2 Architecture of Proposed System

Architecture Overview:

1. DigiWizard Core:

- Central processing unit handling data processing, communication, and control tasks.

2. Touchscreen Interface:

- Provides a user-friendly interface for interaction.
- Displays controls, settings, and feedback for IR and RFID/NFC functionalities.

3. **IR Emulation Module:**

- Includes an infrared LED and associated circuitry for transmitting IR signals.
- Interfaces with the core processing unit to send commands to electronic devices.

4. **RFID/NFC Emulation Module:**

- RFID/NFC reader/writer for communication with RFID/NFC tags.
- Supports multiple protocols and standards for versatile applications.

5. **Power Supply Unit:**

- Ensures stable and reliable power delivery to **DigiWizard** and connected components.

6. **Communication Interfaces:**

- Wi-Fi and Bluetooth modules for wireless connectivity and integration with smart home devices.

7. **Software Components:**

- Operating system, libraries, and application software for managing system operations and user interactions.

4.2.1 Architecture Diagram:

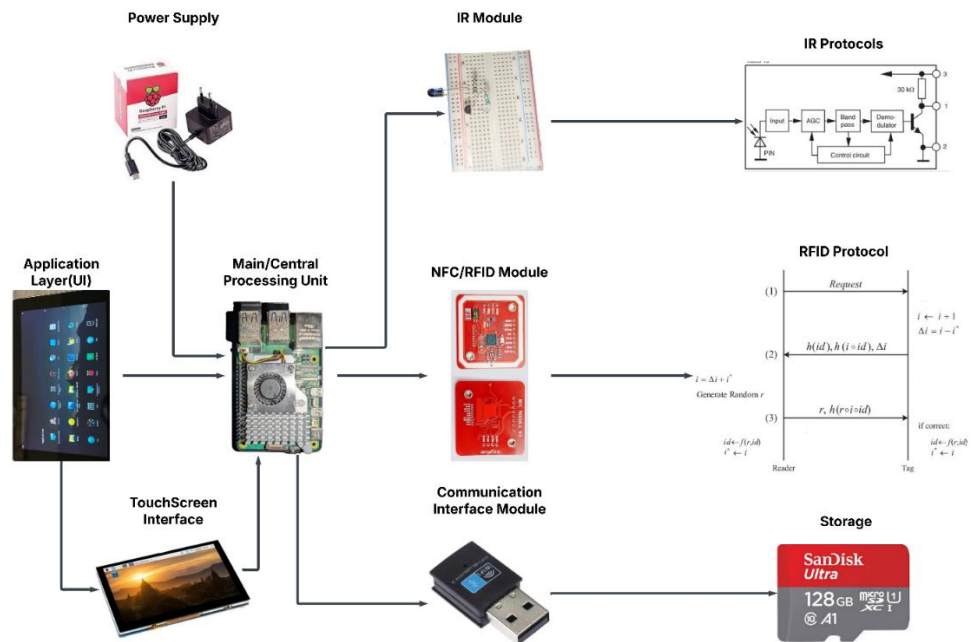


Fig. 4.2.1. Architecture diagram

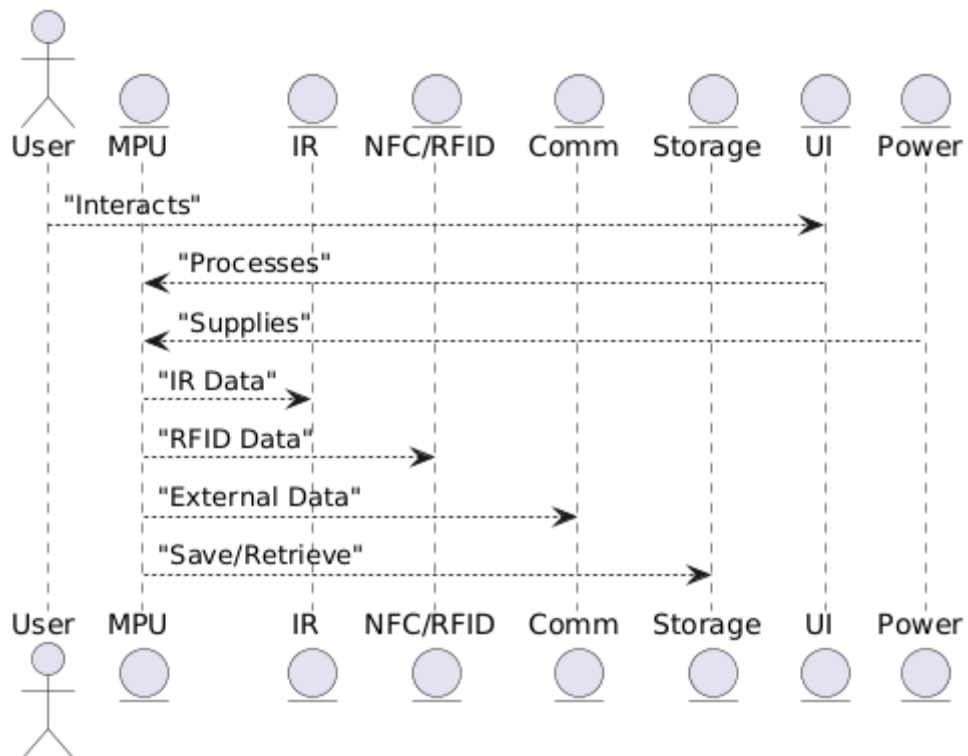


Fig. 4.2.2. Data flow Diagram

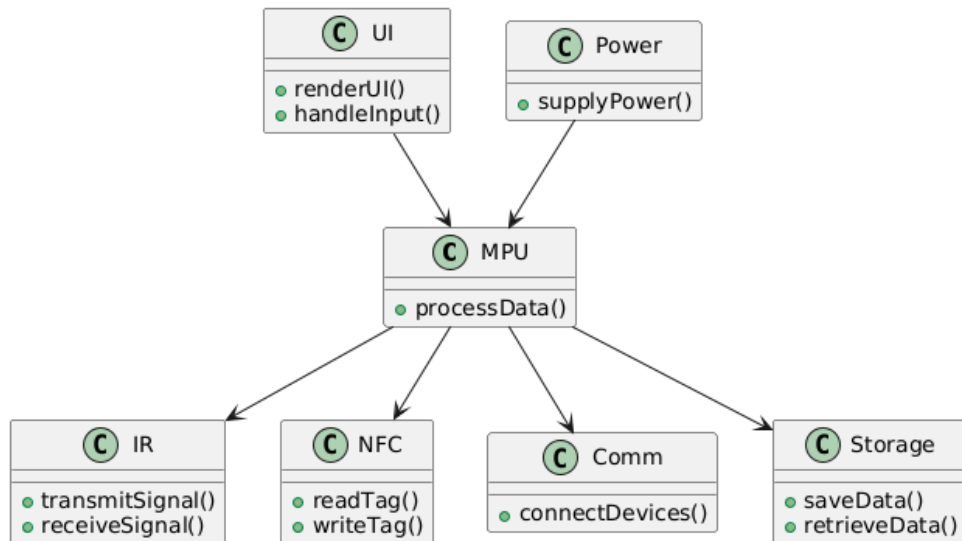


Fig. 4.2.3. Class Diagram

4.3 Detailed Description of Modules and Workflow

IR Emulation Module

1. Component Overview:

- **Infrared LED:** Emits IR signals to control electronic devices.
- **GPIO Interface:** Connects the IR LED to **DigiWizard** for signal transmission.

2. Functionality:

- Users select the desired device and command from the touchscreen interface.
- **DigiWizard** generates the corresponding IR signal and sends it to the IR LED.
- The LED transmits the signal, controlling the target device.

3. Workflow:

- **User Interaction:** User selects device and command via the touchscreen.

- **Signal Generation:** **DigiWizard** processes the input and generates the IR signal.
- **Transmission:** IR LED transmits the signal to the target device.
- **Feedback:** System provides visual feedback on successful command execution.

RFID/NFC Emulation Module

1. Component Overview:

- **RFID/NFC Reader/Writer:** Supports communication with RFID/NFC tags.
- **I2C/SPI Interface:** Connects the module to **DigiWizard** for data exchange.

2. Functionality:

- Enables secure access control and payment applications by emulating RFID/NFC tags.
- Supports reading and writing data to RFID/NFC tags for versatile applications.

3. Workflow:

- **User Interaction:** User selects RFID/NFC application via the touchscreen.
- **Tag Emulation:** **DigiWizard** configures the reader/writer to emulate the desired tag.
- **Communication:** Reader/writer interacts with RFID/NFC tags for data exchange.
- **Feedback:** System provides confirmation and feedback on successful operations.

Touchscreen Interface Module

1. Component Overview:

- **Display Screen:** Provides an interactive interface for user interactions.
- **Touch Controller:** Detects and processes touch inputs for user commands.

2. Functionality:

- Displays system controls, settings, and feedback for IR and RFID/NFC functionalities.
- Allows users to navigate menus, select options, and execute commands.

3. Workflow:

- **User Interaction:** User interacts with the touchscreen to access system features.
- **Command Processing:** **DigiWizard** processes touch inputs and executes corresponding actions.
- **Visual Feedback:** Display provides real-time feedback on system operations and status.

Communication and Integration Module

1. Component Overview:

- **Wi-Fi and Bluetooth Modules:** Enable wireless communication and integration with smart devices.

2. Functionality:

- Facilitates connectivity with smart home devices and external systems.
- Supports data exchange and remote-control features.

3. Workflow:

- **Connection Setup: DigiWizard** establishes connections with compatible devices.
- **Data Exchange:** System communicates with external devices for control and data transfer.
- **Feedback:** Provides status updates and notifications on connectivity and operations.

4.4 Implementation of Existing System:

Phase 1:

- Started with plain raspberry pi and without any modules and worked on the installation packages for LTS.
- Changed the stream of OS from Release to Release-LTS

Phase 2:

- Started the IR integration by using pre-built IR module.
- Used LIRC protocol and bare LIRC commands for the IR.

Phase 3:

- Started the Custom IR module by using TSOP38238 as the IR receiver.
- Introduced the apt package "ir-keytable" which by default uses LIRC.
- Introduced the idea of RFID integration

Phase 4:

- For RFID, we are going to use MFRC522 module and the python package "simplemfr522".
- Make a small program to test the working of MFRC module.
- For IR, along with checked the working of ir-keytable along with the LIRC Module

Phase 5:

- Started to research for the GUI for a modern tablet
- Started with bash scripts for the basic automation like updating the system, etc.

Phase 6:

- Replaced TSOP38238 with VS1838B for better signal handling
- Replaced bash scripts with Python scripts for GUI and easy working

Phase 7:

- Replaced MFRC522 module with PN532 module for RFID as well as NFC functions
- Replaced the MFRC522 library with libnfc-bin for PN532 module
- Created a tablet-based GUI with KDE-Plasma & X11 Desktop environment

Phase 8:

- Created a custom IR emitter using:
 1. NPN Transistor PN2222
 2. 1k ohm resistor
 3. 220 ohm resistor
 4. IR Led

Phase 9:

- Connected 7" DSI display with raspberry pi
- Extended 40 pin GPIO.
- Completed all the GUI part along with update and minor scripts

4.5 Financial Report on Estimated Costing

HARDWARE COMPONENTS	DESCRIPTION	ESTIMATION COST(INR)
RASPBERRY PI 5	4GB RAM, INCLUDES POWER SUPPLY AND CASE	6500
7" TOUCH SCREEN DISPLAY	7 INCH DISPLAY WITH TOUCH SCREEN CAPABILITIES	7000
RFID/NFC READER/WRITER MODULE	PN532 MODULE	400
IR TRANSMITTER MODULE	INFRARED LED WITH NECESSARY COMPONENTS	300
CABLES AND CONNECTORS	HDMI, USB CABLES AND GPIO CONNECTORS	~ 800

Table 4.5.1 Hardware Components Expenses

SOFTWARE COMPONENTS	DESCRIPTION	ESTIMATION PRICE(INR)
RASPBERRY PI OS	OPEN SOURCE	0
PYTHON PROGRAMMING LIBRARIES	OPEN SOURCE	0
GUI FRAMEWORKS	OPEN SOURCE	0
SECURITY TOOLS	OPEN SOURCE	0

Table 4.5.2. Software Components Expenses

OVERHEAD CATEGORY	DESCRIPTION	ESTIMATION PRICE(INR)
IDE AND SOFTWARES	SOFTWARE LICENCES (IF ANY)	1000-2000
TESTING EQUIPMENTS	MULTIMETER, ETC	2000-3000
PROTOTYPING MATERIALS	BREADBOARD, JUMPER WIRES, RESISTORS, ETC	1000-2000
MISCALLENIOUS	EXTRA COMPONENTS, SPARE COMPONENTS, SOLDERING KIT	1000
SHIPPING AND HANDLING	COST OF SHIPPING FOR PARTS AND TOOLS	1000-2000
DOCUMENTATION AND PRINTING	COST OF PRINTING MANUALS AND PROJECT REPORT	500
CONTINGENCY FUND	BUDGET FOR UNFORESEEN EXPENSES	1000-2000

Table 4.5.3. Overhead Expenses

Total Estimated Cost

- Total Hardware Cost: ₹ 15,000
- Total Software Cost: ₹0
- Total Overhead Cost: ₹10,000
- Grand Total Cost: ₹25,000

CHAPTER 5

RESULTS AND DISCUSSION

5.1. RESULTS

The project was successfully designed, developed, and tested to meet the intended functionalities. The Raspberry Pi-based tablet integrates IR and RFID/NFC emulation, allowing seamless interaction with various electronic devices and smart access systems. The system effectively processes user inputs and transmits signals accurately, ensuring reliable performance.

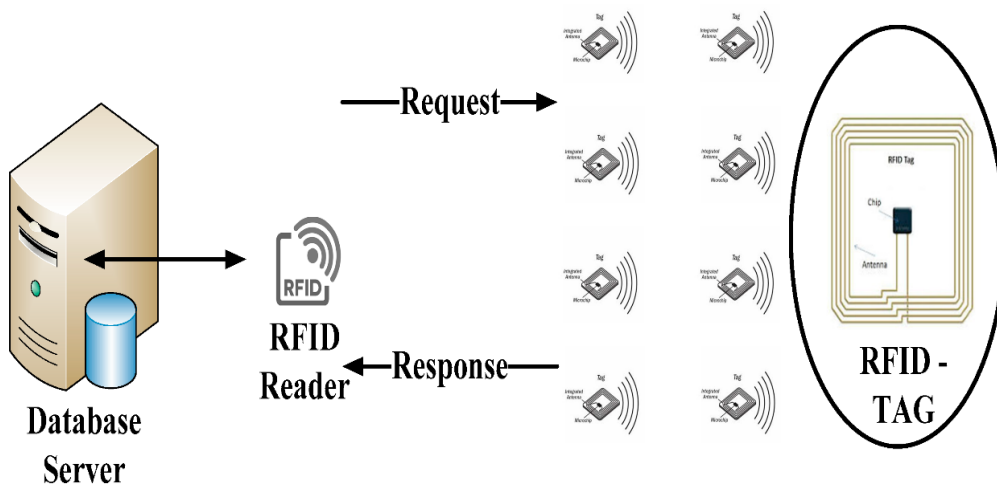


Fig. 5.1: RFID/NFC authentication process flow

The IR transmitter is designed to function as a universal remote control, allowing users to interact with multiple infrared-based appliances. The RFID/NFC module accurately reads and emulates tags, enabling secure authentication and access control. The touchscreen interface ensures an intuitive user experience, making navigation and configuration easy.

Additionally, the system logs user interactions and maintains a secure record of authentication attempts, which can be used for monitoring and debugging. The data processing speed and response time were tested under various conditions, demonstrating consistent performance across different applications.

Condition	Obstacle Material	Distance (cm)	Response Time (ms)	Detection Accuracy (%)
Normal	Cardboard	5	50	95
Normal	Plastic	5	45	96
Increased	Cardboard	10	100	90
Increased	Plastic	10	90	89
Obstructed	Glass	5	150	85
Obstructed	Metal Sheet	5	180	80
Bright Light	Cardboard	5	200	72
Bright Light	Plastic	5	180	78

Table 5.1.1 Response Time of IR Sensor

5.2. FUTURE ENCHANCEMENTS

Future improvements could focus on enhancing the system's performance, security, and connectivity:

- **Security Enhancements:** Implementing **encryption for RFID/NFC data** to improve privacy and prevent unauthorized access.
- **Power Optimization:** Integrating **low-power modes** to extend battery life and improve energy efficiency.
- **Expanded Connectivity:** Adding support for **Wi-Fi and Bluetooth** to enable remote access and cloud integration.
- **Advanced User Interface:** Refining the **GUI for better accessibility**, including voice command support and gesture recognition.
- **Integration with IoT Platforms:** Enabling **real-time data logging and analytics** by connecting to cloud-based IoT services.

These enhancements would make the Raspberry Pi-based tablet even more versatile, broadening its applications in **smart home automation, industrial control, and secure authentication systems**.

CHAPTER 6

CONCLUSION

The **DigiWizard** project successfully integrates multiple technologies into a unified system, demonstrating the potential of IoT-driven automation. By leveraging Raspberry Pi, IR, and RFID/NFC emulation, **DigiWizard** enhances digital accessibility, secure authentication, and seamless device control. The incorporation of a touchscreen interface and a user-friendly GUI ensures an intuitive experience for users, making the system adaptable across various domains, including smart homes and industrial automation. This project highlights the versatility of open-source hardware in creating innovative, cost-effective solutions for modern technological needs.

Throughout development, challenges such as component integration, real-time processing, and power efficiency were encountered, offering valuable insights into optimizing embedded systems. The modular nature of **DigiWizard** allows for future enhancements, including stronger encryption for secure communication, expanded compatibility with additional IoT devices, and improved connectivity options. As technology continues to evolve, this project serves as a foundation for advancing digital automation, illustrating how embedded systems can drive innovation in security, access control, and smart technology solutions.

REFERENCES

- [1] Adams, James (2016): Discusses the technical advancements in the Raspberry Pi 3B+, including its increased processing power, integrated Wi-Fi and Bluetooth capabilities, and expanded GPIO functionalities. This work outlines the features that make the Raspberry Pi suitable for complex projects like this one.
- [2] Chaudhary, R., & Singh, N. (2021): "Exploring the Security Implications of RFID and NFC Technologies," in *Information Security Journal*, 30(2),* 100-115. This research explores the security aspects of RFID and NFC, providing an in-depth analysis of potential vulnerabilities and mitigation strategies.
- [3] Dominguez et al. (2018): Discuss the integration of RFID technology in Raspberry Pi projects for automated inventory management systems. Their research demonstrates the Raspberry Pi's ability to read and write RFID tags, highlighting its versatility as a data-gathering tool.
- [4] Eben Upton et al. (2012): In their foundational work on the Raspberry Pi, they describe the motivation behind its creation and its potential impact on computer science education. The device was initially intended to address the declining numbers of students applying for computer science courses at the University of Cambridge, highlighting its educational origins.
- [5] Gupta, P., Kumar, A., & Bharti, P. (2020): "Raspberry Pi-Based Embedded Systems: Applications and Future Directions," in *International Journal of Advanced Computer Science and Applications*, 11(5),* 450-459.
- [6] Harrington and Haverbeke (2017): Explore using Raspberry Pi in home automation projects, focusing on IR-based control systems. Their work highlights the ability of the Raspberry Pi to act as a universal remote control through IR code libraries and customizable scripts.

[7] Kumar, A., & Raj, N. (2021): Explore the development of educational tools using Raspberry Pi, focusing on integrating various communication technologies, including IR and RFID/NFC. Their work emphasizes the importance of such projects in teaching electronics and programming concepts.

[8] Lee, H., & Kim, S. (2019): "Utilizing Raspberry Pi in IoT-Based Automation Systems: A Review," in *IEEE Access, 7,* 2000-2015. This review discusses the role of Raspberry Pi in IoT automation systems, emphasizing its capabilities in handling diverse protocols and integration scenarios.

[9] Mansfield et al. (2019): Present a project integrating IR and RFID technologies in a home automation system, demonstrating the combined use of these technologies to enhance control and security. Their research illustrates the potential of such integrations in creating comprehensive smart home solutions.

[10] Monk, Simon (2014): In his book "Raspberry Pi Cookbook," demonstrates practical examples of using the Raspberry Pi to send and receive IR signals. The book provides insights into setting up IR circuits and utilizing Python scripts to emulate remote control functions.

[11] NFC Forum (2020): Provides comprehensive guidelines on implementing NFC technology, focusing on the secure exchange of data and applications in contactless payment systems. This work is crucial for understanding the potential applications of NFC in secure transactions.

[12] Smith, J., & Chan, T. (2018): "Next-Generation Infrared Technology: Innovations and Applications," in *Journal of Optical Communications, 40(3),* 120-132. This paper examines cutting-edge innovations in IR technology, highlighting the increased range and data throughput capabilities.

[13] Wong, L., & Lin, C. (2017): "The Evolution of IR Technology in Consumer Electronics," in *Consumer Electronics Review, 14(4),* 245-259. This work reviews advancements in IR technology, focusing on its integration with smart devices.

APPENDIX

A. SOURCE CODE

Complete script for ir detection in PyQt5: (Python)

```
#!/usr/bin/env python3

import sys
import os
import subprocess
from PyQt5.QtWidgets import (QApplication, QMainWindow, QVBoxLayout,
                              QHBoxLayout,
                              QPushButton, QTextEdit, QLabel, QWidget, QMessageBox,
                              QComboBox, QGroupBox)
from PyQt5.QtCore import Qt, QProcess

class IRKeyTableGUI(QMainWindow):
    def __init__(self):
        super().__init__()
        self.initUI()
        self.process = None
        self.check_dependencies()

    def check_dependencies(self):
        missing_deps = []

        if subprocess.run(["which", "ir-keytable"],
                          stdout=subprocess.PIPE, stderr=subprocess.PIPE).returncode !=
0:
            missing_deps.append("ir-keytable (install v4l-utils package)")

        self.terminal_cmd = None
        terminals = [
            ("lxterminal", ["-e"]),
            ("konsole", ["-e"]),
            ("gnome-terminal", ["--"]),
            ("xterm", ["-e"]),
            ("xfce4-terminal", ["-e"]),
            ("terminator", ["-e"]),
            ("mate-terminal", ["-e"])
        ]

        for term, args in terminals:
            if subprocess.run(["which", term],
```

```

        stdout=subprocess.PIPE, stderr=subprocess.PIPE).returncode
== 0:
    self.terminal_cmd = term
    self.terminal_args = args
    break

    if not self.terminal_cmd:
        missing_deps.append("Terminal emulator (lxterminal, konsole, gnome-
terminal, xterm, etc.)")

    if missing_deps:
        msg = "The following dependencies are missing:\n• " + "\n•
".join(missing_deps)
        QMessageBox.warning(self, "Missing Dependencies", msg)
        self.output_display.append(msg)
        self.probe_button.setEnabled(False)
        self.test_button.setEnabled(False)
    else:
        self.output_display.append(f"Using terminal: {self.terminal_cmd}")

def initUI(self):
    self.setWindowTitle("IR Keytable Tester")
    self.setGeometry(100, 100, 600, 400)

    central_widget = QWidget()
    self.setCentralWidget(central_widget)
    main_layout = QVBoxLayout(central_widget)

    title_label = QLabel("IR Remote Control Tester")
    title_label.setAlignment(Qt.AlignCenter)
    font = title_label.font()
    font.setPointSize(14)
    font.setBold(True)
    title_label.setFont(font)
    main_layout.addWidget(title_label)

    protocol_group = QGroupBox("IR Protocol")
    protocol_layout = QVBoxLayout()

    self.protocol_combo = QComboBox()
    self.protocol_combo.addItem("All Protocols", "all")
    self.protocol_combo.addItem("RC-5", "rc-5")
    self.protocol_combo.addItem("RC-6", "rc-6")
    self.protocol_combo.addItem("NEC", "nec")
    self.protocol_combo.addItem("JVC", "jvc")
    self.protocol_combo.addItem("Sony", "sony")
    protocol_layout.addWidget(self.protocol_combo)

```

```

protocol_group.setLayout(protocol_layout)
main_layout.addWidget(protocol_group)

self.output_display = QTextEdit()
self.output_display.setReadOnly(True)
main_layout.addWidget(self.output_display)

button_layout = QHBoxLayout()

self.probe_button = QPushButton("Probe IR Protocols")
self.probe_button.clicked.connect(self.probe_protocols)
button_layout.addWidget(self.probe_button)

self.test_button = QPushButton("Test IR Input in Terminal")
self.test_button.clicked.connect(self.launch_terminal_test)
button_layout.addWidget(self.test_button)

self.clear_button = QPushButton("Clear Output")
self.clear_button.clicked.connect(self.clear_output)
button_layout.addWidget(self.clear_button)

main_layout.addLayout(button_layout)

self.status_label = QLabel("Ready")
self.status_label.setAlignment(Qt.AlignCenter)
main_layout.addWidget(self.status_label)

instructions = QLabel(
    "1. Click 'Probe IR Protocols' to detect supported protocols\n"
    "2. Click 'Test IR Input in Terminal' to open a terminal window\n"
    "3. In the terminal, press keys on your remote to see detected signals\n"
    "4. Press Ctrl+C in the terminal to exit the test"
)
instructions.setAlignment(Qt.AlignCenter)
main_layout.addWidget(instructions)

def clear_output(self):
    self.output_display.clear()

def probe_protocols(self):
    protocol = self.protocol_combo.currentData()
    self.output_display.clear()
    self.output_display.append(f"Probing {protocol} IR protocols...\n")
    self.status_label.setText(f"Probing {protocol} IR protocols...")

self.process = QProcess()

```

```

self.process.readyReadStandardOutput.connect(self.handle_stdout)
self.process.readyReadStandardError.connect(self.handle_stderr)
self.process.finished.connect(self.handle_probe_finished)

try:
    cmd = "ir-keytable"
    if os.geteuid() != 0:
        cmd = "pkexec " + cmd

    self.process.start(cmd, ["-p", protocol])
except Exception as e:
    self.output_display.append(f"Error: {str(e)}")
    self.status_label.setText("Error starting ir-keytable command")

def launch_terminal_test(self):
    if not self.terminal_cmd:
        QMessageBox.warning(
            self,
            "Terminal Not Found",
            "Could not find a suitable terminal emulator."
        )
    return

def handle_stdout(self):
    data = self.process.readAllStandardOutput().data().decode('utf-8')
    self.output_display.append(data.strip())

def handle_stderr(self):
    data = self.process.readAllStandardError().data().decode('utf-8')
    self.output_display.append("ERROR: " + data.strip())

def handle_probe_finished(self, exit_code, exit_status):
    if exit_code == 0:
        self.status_label.setText("Protocol probe completed successfully")
    else:
        self.status_label.setText(f"Protocol probe failed with exit code {exit_code}")

def closeEvent(self, event):
    if self.process and self.process.state() == QProcess.Running:
        self.process.kill()
    event.accept()

def main():
    app = QApplication(sys.argv)

    try:

```

```

        from PyQt5.QtWidgets import QStyleFactory
        if "Breeze" in QStyleFactory.keys():
            app.setStyle("Breeze")
    except ImportError:
        pass

    window = IRKeyTableGUI()
    window.show()
    sys.exit(app.exec_())

if __name__ == "__main__":
    main()

```

Script to install required dependencies: (Bash)

```

#!/bin/bash

set -e

# Ensure the script is run as root
if [ "$(id -u)" -ne 0 ]; then
    echo "Please run this script as root (use sudo)."
```

 exit 1
fi

Update and upgrade the system
echo "Updating system packages..."
apt update && apt upgrade -y

Install required packages
echo "Installing required packages: Python3, PyQt5, and v4l-utils..."
apt install -y python3 python3-pyqt5 v4l-utils

Ensure ir-keytable is available
if ! command -v ir-keytable &> /dev/null; then
 echo "Error: ir-keytable installation failed."
 exit 1
fi

Ensure a terminal emulator is available
TERMINAL=\$(which lxterminal || which xterm || which gnome-terminal || which
konsole || which xfce4-terminal || which terminator || which mate-terminal)
if [-z "\$TERMINAL"]; then
 echo "No suitable terminal emulator found. Installing lxterminal..."
 apt install -y lxterminal

```

fi

# Confirm the environment setup
echo "Setup complete! You can now run the IR Keytable GUI."
echo "Usage: python3 ir_keytable_gui.py"

exit 0

```

Desktop application script: (Bash)

```

#!/bin/bash

set -e

echo "Installing IR Keytable Tester Application..."

echo "Checking for required packages..."
MISSING_PACKAGES=""

if ! python3 -c "import PyQt5" &>/dev/null; then
    MISSING_PACKAGES+=" python3-pyqt5"
fi

if ! which ir-keytable &>/dev/null; then
    MISSING_PACKAGES+=" v4l-utils"
fi

if ! which pkexec &>/dev/null; then
    MISSING_PACKAGES+=" policykit-1"
fi

if [ -n "$MISSING_PACKAGES" ]; then
    echo "The following packages need to be installed: $MISSING_PACKAGES"

    if which apt &>/dev/null; then
        sudo apt update
        sudo apt install -y $MISSING_PACKAGES
    elif which dnf &>/dev/null; then
        sudo dnf install -y $MISSING_PACKAGES
    elif which pacman &>/dev/null; then
        sudo pacman -S --needed $MISSING_PACKAGES
    else
        echo "ERROR: Could not detect package manager. Please install manually:
$MISSING_PACKAGES"
        exit 1
    fi
fi

```

```
fi
fi
```

```
echo "Creating application directories..."
mkdir -p ~/.local/bin
mkdir -p ~/.local/share/applications
mkdir -p ~/.local/share/icons/hicolor/scalable/apps
```

```
echo "Creating application icon..."
cat > ~/.local/share/icons/hicolor/scalable/apps/ir-keytable-tester.svg << 'EOL'
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 100 100">
  <circle cx="50" cy="50" r="45" fill="#3498db" />
  <rect x="35" y="20" width="30" height="60" rx="5" fill="#2c3e50" />
  <circle cx="50" cy="35" r="5" fill="#e74c3c" />
  <rect x="40" y="45" width="20" height="5" rx="2" fill="#ecf0f1" />
  <rect x="40" y="55" width="20" height="5" rx="2" fill="#ecf0f1" />
  <rect x="40" y="65" width="20" height="5" rx="2" fill="#ecf0f1" />
  <path d="M65 35 Q75 25, 85 35" fill="none" stroke="#ecf0f1" stroke-width="2" />
  <path d="M65 40 Q80 30, 90 40" fill="none" stroke="#ecf0f1" stroke-width="2" />
  <path d="M65 30 Q70 20, 80 30" fill="none" stroke="#ecf0f1" stroke-width="2" />
</svg>
EOL
```

```
echo "Installing the application script..."
cat > ~/.local/bin/ir-keytable-tester.py << 'EOL'
#!/usr/bin/env python3
```

```
import sys
import os
import subprocess
from PyQt5.QtWidgets import QApplication, QMainWindow, QLabel
```

```
class IRKeyTableGUI(QMainWindow):
    def __init__(self):
        super().__init__()
        self.initUI()

    def initUI(self):
        self.setWindowTitle("IR Keytable Tester")
        self.setGeometry(100, 100, 400, 300)
        label = QLabel("IR Keytable Tester GUI", self)
        label.setGeometry(100, 130, 200, 40)

        if not self.check_dependencies():
            label.setText("Missing Dependencies")
```

```

def check_dependencies(self):
    required = ["ir-keytable", "pkexec"]
    for cmd in required:
        if subprocess.run(["which", cmd], stdout=subprocess.PIPE).returncode !=
0:
            return False
    return True

def main():
    app = QApplication(sys.argv)
    window = IRKeyTableGUI()
    window.show()
    sys.exit(app.exec_())

if __name__ == "__main__":
    main()
EOL

```

```

chmod +x ~/.local/bin/ir-keytable-tester.py

```

```

echo "Creating desktop entry..."
cat > ~/.local/share/applications/ir-keytable-tester.desktop << EOL
[Desktop Entry]
Version=1.0
Type=Application
Name=IR Keytable Tester
Comment=Test and debug infrared remote controls
Exec=python3 ~/.local/bin/ir-keytable-tester.py
Icon=ir-keytable-tester
Terminal=false
Categories=Utility;System;
Keywords=remote;control;ir;infrared;keytable;
StartupNotify=true
EOL

```

```

POLKIT_DIR="/etc/polkit-1/actions"
POLKIT_FILE="org.irkeytable.pkexec.policy"

```

```

cat > /tmp/$POLKIT_FILE << EOL
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE policyconfig PUBLIC "-//freedesktop//DTD PolicyKit Policy
Configuration 1.0//EN"
"http://www.freedesktop.org/standards/PolicyKit/1/policyconfig.dtd">
<policyconfig>
  <action id="org.irkeytable.pkexec">
    <description>Run IR-Keytable with elevated privileges</description>
    <message>Authentication is required to access infrared devices</message>

```

```

<icon_name>ir-keytable-tester</icon_name>
<defaults>
  <allow_any>auth_admin</allow_any>
  <allow_inactive>auth_admin</allow_inactive>
  <allow_active>auth_admin_keep</allow_active>
</defaults>
<annotate key="org.freedesktop.policykit.exec.path">/usr/bin/ir-
keytable</annotate>
</action>
</policyconfig>
EOL

```

```

if sudo mkdir -p $POLKIT_DIR && sudo cp /tmp/$POLKIT_FILE $POLKIT_DIR/
&& sudo chmod 644 $POLKIT_DIR/$POLKIT_FILE; then
  echo "PolicyKit policy installed successfully."
else
  echo "Could not install PolicyKit policy."
fi

```

```

if which gtk-update-icon-cache &>/dev/null; then
  gtk-update-icon-cache -f -t ~/.local/share/icons/hicolor
fi

```

```

echo "Installation complete!"
echo "Run the application: python3 ~/.local/bin/ir-keytable-tester.py"

```

LIRC protocol by ir-keytable APT package:

rc_map.conf file:

(.conf)

```

#
# Keymaps table
#
# This table creates an association between a keycode file and a kernel
# driver. It can be used to automatically override a keycode definition.
#
# Although not yet tested, it is mented to be added at udev.
#
# To use, you just need to run:
# ./ir-keytable -a
#
# Or, if the remote is not the first device:
# ./ir-keytable -a -s rc1          # for RC at rc1
#

```

Format:

```

# driver - name of the driver provided via uevent - use * for any driver
# table - RC keymap table, provided via uevent - use * for any table
# file - file name. If directory is not specified, it will default to
# /etc/rc_keymaps.
# For example:
# driver    table                                file
# cx8800    *                                  ./keycodes/rc5_hauppage_new.toml
# *          rc-avermedia-m135a-rm-jx ./keycodes/kworld_315u.toml
# saa7134 rc-avermedia-m135a-rm-jx
#           ./keycodes/keycodes/nec_terratec_cinergy_xs.toml
# em28xx    *                                  ./keycodes/kworld_315u.toml
# *          *                                  ./keycodes/rc5_hauppage_new.toml

# Table to automatically load the rc maps for the bundled IR's provided with the
# devices supported by the linux kernel

```

```

#driver table    file
* rc-adstech-dvb-t-pci  adstech_dvb_t_pci.toml
* rc-alink-dtu-m        alink_dtu_m.toml
* rc-anysee             anysee.toml
* rc-apac-viewcomp      apac_viewcomp.toml
* rc-astrometa-t2hybrid  astrometa_t2hybrid.toml
* rc-asus-pc39           asus_pc39.toml
* rc-asus-ps3-100        asus_ps3_100.toml
* rc-ati-tv-wonder-hd-600 ati_tv_wonder_hd_600.toml
* rc-ati-x10            ati_x10.toml
* rc-avermedia-a16d      avermedia_a16d.toml
* rc-avermedia-cardbus   avermedia_cardbus.toml
* rc-avermedia-dvbt      avermedia_dvbt.toml
* rc-avermedia-m135a     avermedia_m135a.toml
* rc-avermedia-m733a-rm-k6 avermedia_m733a_rm_k6.toml
* rc-avermedia-rm-ks     avermedia_rm_ks.toml
* rc-avermedia           avermedia.toml
* rc-avertv-303          avertv_303.toml
* rc-azurewave-ad-tu700  azurewave_ad_tu700.toml
* rc-beelink-gs1         beelink_gs1.toml
* rc-behold-columbus     behold_columbus.toml
* rc-behold              behold.toml
* rc-budget-ci-old       budget_ci_old.toml
* rc-cec                 cec.toml
* rc-cinergy-1400        cinergy_1400.toml
* rc-cinergy             cinergy.toml
* rc-ct-90405            ct_90405.toml
* rc-d680-dmb            d680_dmb.toml
* rc-delock-61959        delock_61959.toml
* rc-dib0700-nec         dib0700_nec.toml
* rc-dib0700-rc5         dib0700_rc5.toml

```

- * rc-digitalnow-tinytwin digitalnow_tinytwin.toml
- * rc-digittrade digittrade.toml
- * rc-dm1105-nec dm1105_nec.toml
- * rc-dntv-live-dvb-t dntv_live_dvb_t.toml
- * rc-dntv-live-dvbt-pro dntv_live_dvbt_pro.toml
- * rc-dtt200u dtt200u.toml
- * rc-dvbsky dvbsky.toml
- * rc-dvico-mce dvico_mce.toml
- * rc-dvico-portable dvico_portable.toml
- * rc-em-terratec em_terratec.toml
- * rc-encore-enltv-fm53 encore_enltv_fm53.toml
- * rc-encore-enltv encore_enltv.toml
- * rc-encore-enltv2 encore_enltv2.toml
- * rc-evga-indtube evga_indtube.toml
- * rc-eztv eztv.toml
- * rc-flydvb flydvb.toml
- * rc-flyvideo flyvideo.toml
- * rc-fusionhdtv-mce fusionhdtv_mce.toml
- * rc-gadmei-rm008z gadmei_rm008z.toml
- * rc-geekbox geekbox.toml
- * rc-genius-tvgo-a11mce genius_tvgo_a11mce.toml
- * rc-gotview7135 gotview7135.toml
- * rc-hauppauge hauppauge.toml
- * rc-hisi-poplar hisi_poplar.toml
- * rc-hisi-tv-demo hisi_tv_demo.toml
- * rc-imon-mce imon_mce.toml
- * rc-imon-pad imon_pad.toml
- * rc-imon-rsc imon_rsc.toml
- * rc-iodata-bctv7e iodata_bctv7e.toml
- * rc-it913x-v1 it913x_v1.toml
- * rc-it913x-v2 it913x_v2.toml
- * rc-kaiomy kaiomy.toml
- * rc-khadas khadas.toml
- * rc-khamsin khamsin.toml
- * rc-kworld-315u kworld_315u.toml
- * rc-kworld-pc150u kworld_pc150u.toml
- * rc-kworld-plus-tv-analog kworld_plus_tv_analog.toml
- * rc-leadtek-y04g0051 leadtek_y04g0051.toml
- * rc-lme2510 lme2510.toml
- * rc-manli manli.toml
- * rc-mecool-kii-pro mecool_kii_pro.toml
- * rc-mecool-kiii-pro mecool_kiii_pro.toml
- * rc-medion-x10-digitainer medion_x10_digitainer.toml
- * rc-medion-x10-or2x medion_x10_or2x.toml
- * rc-medion-x10 medion_x10.toml
- * rc-minix-neo minix_neo.toml
- * rc-msi-digivox-ii msi_digivox_ii.toml

- * rc-msi-digivox-iii msi_digivox_iii.toml
- * rc-msi-tvanywhere-plus msi_tvanywhere_plus.toml
- * rc-msi-tvanywhere msi_tvanywhere.toml
- * rc-nebula nebula.toml
- * rc-nec-terratec-cinergy-xs nec_terratec_cinergy_xs.toml
- * rc-norwood norwood.toml
- * rc-npgtech npgtech.toml
- * rc-odroid odroid.toml
- * rc-pctv-sedna pctv_sedna.toml
- * rc-pine64 pine64.toml
- * rc-pinnacle-color pinnacle_color.toml
- * rc-pinnacle-grey pinnacle_grey.toml
- * rc-pinnacle-pctv-hd pinnacle_pctv_hd.toml
- * rc-pixelview-002t pixelview_002t.toml
- * rc-pixelview-mk12 pixelview_mk12.toml
- * rc-pixelview-new pixelview_new.toml
- * rc-pixelview pixelview.toml
- * rc-powercolor-real-angel powercolor_real_angel.toml
- * rc-proteus-2309 proteus_2309.toml
- * rc-purpletv purpletv.toml
- * rc-pv951 pv951.toml
- * rc-rc6-mce rc6_mce.toml
- * rc-real-audio-220-32-keys real_audio_220_32_keys.toml
- * rc-reddo reddo.toml
- * rc-snapstream-firefly snapstream_firefly.toml
- * rc-streamzap streamzap.toml
- * rc-su3000 su3000.toml
- * rc-tanix-tx3mini tanix_tx3mini.toml
- * rc-tanix-tx5max tanix_tx5max.toml
- * rc-tbs-nec tbs_nec.toml
- * rc-technisat-ts35 technisat_ts35.toml
- * rc-technisat-usb2 technisat_usb2.toml
- * rc-terratec-cinergy-c-pci terratec_cinergy_c_pci.toml
- * rc-terratec-cinergy-s2-hd terratec_cinergy_s2_hd.toml
- * rc-terratec-cinergy-xs terratec_cinergy_xs.toml
- * rc-terratec-slim-2 terratec_slim_2.toml
- * rc-terratec-slim terratec_slim.toml
- * rc-tevii-nec tevii_nec.toml
- * rc-tivo tivo.toml
- * rc-total-media-in-hand-02 total_media_in_hand_02.toml
- * rc-total-media-in-hand total_media_in_hand.toml
- * rc-trekstor trekstor.toml
- * rc-tt-1500 tt_1500.toml
- * rc-twinhan-dtv-cab-ci twinhan_dtv_cab_ci.toml
- * rc-twinhan1027 twinhan_vp1027_dvbs.toml
- * rc-vega-s9x vega_s9x.toml
- * rc-videomate-k100 videomate_k100.toml

```

* rc-videomate-s350      videomate_s350.toml
* rc-videomate-tv-pvr    videomate_tv_pvr.toml
* rc-videostrong-kii-pro kii_pro.toml
* rc-wetek-hub           wetek_hub.toml
* rc-wetek-play2         wetek_play2.toml
* rc-winfast-usbii-deluxe winfast_usbii_deluxe.toml
* rc-winfast             winfast.toml
* rc-x96max              x96max.toml
* rc-xbox-dvd            xbox_dvd.toml
* rc-zx-irdec            zx_irdec.toml
# * *                  af9005.toml      # found in af9005-remote.c
# * *                  az6027.toml     # found in az6027.c
# * *                  cinergyt2.toml  # found in cinergyT2-core.c
# * *                  dibusb.toml     # found in dibusb-common.c
# * *                  digitv.toml     # found in digitv.c
# * *                  megasky.toml    # found in m920x.c
# * *                  tvwalkertwin.toml # found in m920x.c
# * *                  pinnacle310e.toml # found in m920x.c
# * *                  haupp.toml      # found in nova-t-usb2.c
# * *                  opera1.toml     # found in opera1.c
# * *                  vp702x.toml     # found in vp702x.c

```

/boot/firmware/config.txt file: (Text)

```

# For more options and information see
# http://rptl.io/configtxt
# Some settings may impact device functionality. See link above for details

# Uncomment some or all of these to enable the optional hardware interfaces
dtparam=i2c_arm=on
#dtparam=i2s=on
dtparam=spi=on

# Enable audio (loads snd_bcm2835)
dtparam=audio=on

# Additional overlays and parameters are documented
# /boot/firmware/overlays/README

# Automatically load overlays for detected cameras
camera_auto_detect=1

# Automatically load overlays for detected DSI displays
display_auto_detect=1

```

```
# Automatically load initramfs files, if found
auto_initramfs=1

# Enable DRM VC4 V3D driver
dtoverlay=vc4-kms-v3d
max_framebuffers=2

# Don't have the firmware create an initial video= setting in cmdline.txt.
# Use the kernel's default instead.
disable_fw_kms_setup=1

# Run in 64-bit mode
arm_64bit=1

# Disable compensation for displays with overscan
disable_overscan=1

# Run as fast as firmware / board allows
arm_boost=1
```

```
[cm4]
# Enable host mode on the 2711 built-in XHCI USB controller.
# This line should be removed if the legacy DWC2 controller is required
# (e.g. for USB device mode) or if USB support is not required.
otg_mode=1
```

```
[cm5]
dtoverlay=dwc2,dr_mode=host
```

```
[all]
dtoverlay=vc4-kms-v3d
#DSI1 Use
dtoverlay=vc4-kms-dsi-7inch
#DSI0 Use (Only Pi5/CM4)
#dtoverlay=vc4-kms-dsi-7inch,dsi0
dtoverlay=lirc-rpi
dtoverlay=gpio-ir,gpio_pin=17
#dtoverlay=gpio-ir-tx,gpio_pin=18
```

Desktop environment kernel editing "lightdm.conf" file: (.conf)

```
#
# General configuration
#
# start-default-seat = True to always start one seat if none are defined in the
configuration
```

```

# greeter-user = User to run greeter as
# minimum-display-number = Minimum display number to use for X servers
# minimum-vt = First VT to run displays on
# lock-memory = True to prevent memory from being paged to disk
# user-authority-in-system-dir = True if session authority should be in the system
location
# guest-account-script = Script to be run to setup guest account
# logind-check-graphical = True to on start seats that are marked as graphical by
logind
# log-directory = Directory to log information to
# run-directory = Directory to put running state in
# cache-directory = Directory to cache to
# sessions-directory = Directory to find sessions
# remote-sessions-directory = Directory to find remote sessions
# greeters-directory = Directory to find greeters
# backup-logs = True to move add a .old suffix to old log files when opening new
ones
# dbus-service = True if LightDM provides a D-Bus service to control it
#
[LightDM]
#start-default-seat=true
#greeter-user=lightdm
#minimum-display-number=0
#minimum-vt=7
#lock-memory=true
#user-authority-in-system-dir=false
#guest-account-script=guest-account
#logind-check-graphical=false
#log-directory=/var/log/lightdm
#run-directory=/var/run/lightdm
#cache-directory=/var/cache/lightdm
#sessions-
directory=/usr/share/lightdm/sessions:/usr/share/xdgsessions:/usr/share/wayland-
sessions
#remote-sessions-directory=/usr/share/lightdm/remote-sessions
#greeters-
directory=$XDG_DATA_DIRS/lightdm/greeters:$XDG_DATA_DIRS/xdgreeters
#backup-logs=true
#dbus-service=true

#
# Seat configuration
#
# Seat configuration is matched against the seat name glob in the section, for
example:
# [Seat:~] matches all seats and is applied first.
# [Seat:seat0] matches the seat named "seat0".

```

```

# [Seat:seat-thin-client*] matches all seats that have names that start with "seat-
thin-client".
#
# type = Seat type (local, xremote, unity)
# pam-service = PAM service to use for login
# pam-autologin-service = PAM service to use for autologin
# pam-greeter-service = PAM service to use for greeters
# xserver-backend = X backend to use (mir)
# xserver-command = X server command to run (can also contain arguments e.g.
X -special-option)
# xmir-command = Xmir server command to run (can also contain arguments e.g.
Xmir -special-option)
# xserver-config = Config file to pass to X server
# xserver-layout = Layout to pass to X server
# xserver-allow-tcp = True if TCP/IP connections are allowed to this X server
# xserver-share = True if the X server is shared for both greeter and session
# xserver-hostname = Hostname of X server (only for type=xremote)
# xserver-display-number = Display number of X server (only for type=xremote)
# xdmcp-manager = XDMCP manager to connect to (implies xserver-allow-
tcp=true)
# xdmcp-port = XDMCP UDP/IP port to communicate on
# xdmcp-key = Authentication key to use for XDM-AUTHENTICATION-1 (stored
in keys.conf)
# unity-compositor-command = Unity compositor command to run (can also
contain arguments e.g. unity-system-compositor -special-option)
# unity-compositor-timeout = Number of seconds to wait for compositor to start
# greeter-session = Session to load for greeter
# greeter-hide-users = True to hide the user list
# greeter-allow-guest = True if the greeter should show a guest login option
# greeter-show-manual-login = True if the greeter should offer a manual login
option
# greeter-show-remote-login = True if the greeter should offer a remote login
option
# user-session = Session to load for users
# allow-user-switching = True if allowed to switch users
# allow-guest = True if guest login is allowed
# guest-session = Session to load for guests (overrides user-session)
# session-wrapper = Wrapper script to run session with
# greeter-wrapper = Wrapper script to run greeter with
# guest-wrapper = Wrapper script to run guest sessions with
# display-setup-script = Script to run when starting a greeter session (runs as
root)
# display-stopped-script = Script to run after stopping the display server (runs as
root)
# greeter-setup-script = Script to run when starting a greeter (runs as root)
# session-setup-script = Script to run when starting a user session (runs as root)
# session-cleanup-script = Script to run when quitting a user session (runs as
root)

```

```

# autologin-guest = True to log in as guest by default
# autologin-user = User to log in with by default (overrides autologin-guest)
# autologin-user-timeout = Number of seconds to wait before loading default user
# autologin-session = Session to load for automatic login (overrides user-session)
# autologin-in-background = True if autologin session should not be immediately
activated
# exit-on-failure = True if the daemon should exit if this seat fails
# fallback-test = Script to run to check if hardware can handle Wayland - should
return 1 if so
# fallback-session = Session to launch if fallback-test returns 0
# fallback-greeter = Greeter session to launch if fallback-test returns 0
#
[Seat:*]
#type=local
#pam-service=lightdm
#pam-autologin-service=lightdm-autologin
#pam-greeter-service=lightdm-greeter
#xserver-backend=
#xserver-command=X
#xmir-command=Xmir
#xserver-config=
#xserver-layout=
#xserver-allow-tcp=false
#xserver-share=true
#xserver-hostname=
#xserver-display-number=
#xdmcp-manager=
#xdmcp-port=177
#xdmcp-key=
#unity-compositor-command=unity-system-compositor
#unity-compositor-timeout=60
greeter-session=pi-greeter-labwc
greeter-hide-users=false
#greeter-allow-guest=true
#greeter-show-manual-login=false
#greeter-show-remote-login=true
user-session=plasma-mobile
#allow-user-switching=true
#allow-guest=true
#guest-session=
#session-wrapper=lightdm-session
#greeter-wrapper=
#guest-wrapper=
display-setup-script=/usr/share/dispsetup.sh
#display-stopped-script=
#greeter-setup-script=
#session-setup-script=

```

```

#session-cleanup-script=
#autologin-guest=false
autologin-user=pi5
#autologin-user-timeout=0
#autologin-in-background=false
autologin-session=plasma-mobile
#exit-on-failure=false
#fallback-test=
#fallback-session=
#fallback-greeter=

#
# XDMCP Server configuration
#
# enabled = True if XDMCP connections should be allowed
# port = UDP/IP port to listen for connections on
# listen-address = Host/address to listen for XDMCP connections (use all
addresses if not present)
# key = Authentication key to use for XDM-AUTHENTICATION-1 or blank to not
use authentication (stored in keys.conf)
# hostname = Hostname to report to XDMCP clients (defaults to system
hostname if unset)
#
# The authentication key is a 56 bit DES key specified in hex as
0xnxxxxxxxxxxxxxxxx. Alternatively
# it can be a word and the first 7 characters are used as the key.
#
[XDMCPServer]
#enabled=false
#port=177
#listen-address=
#key=
#hostname=

#
# VNC Server configuration
#
# enabled = True if VNC connections should be allowed
# command = Command to run Xvnc server with
# port = TCP/IP port to listen for connections on
# listen-address = Host/address to listen for VNC connections (use all addresses
if not present)
# width = Width of display to use
# height = Height of display to use
# depth = Color depth of display to use

```

B. SCREENSHOTS

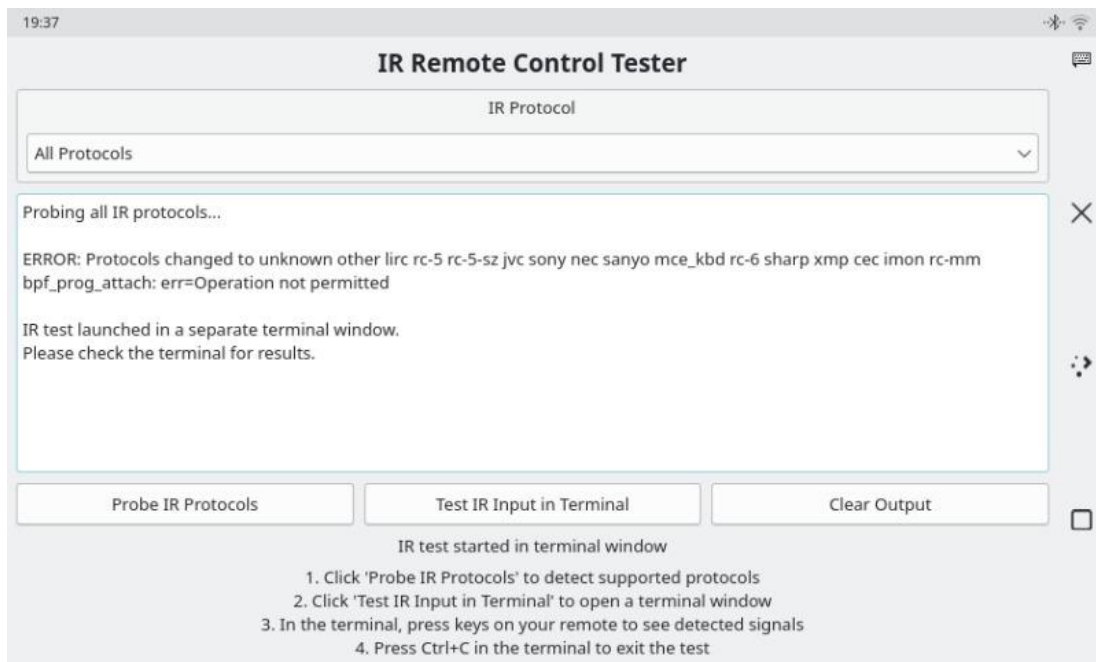


Fig. B.1. Post Execution



Fig. B.2. Initial Script Screen

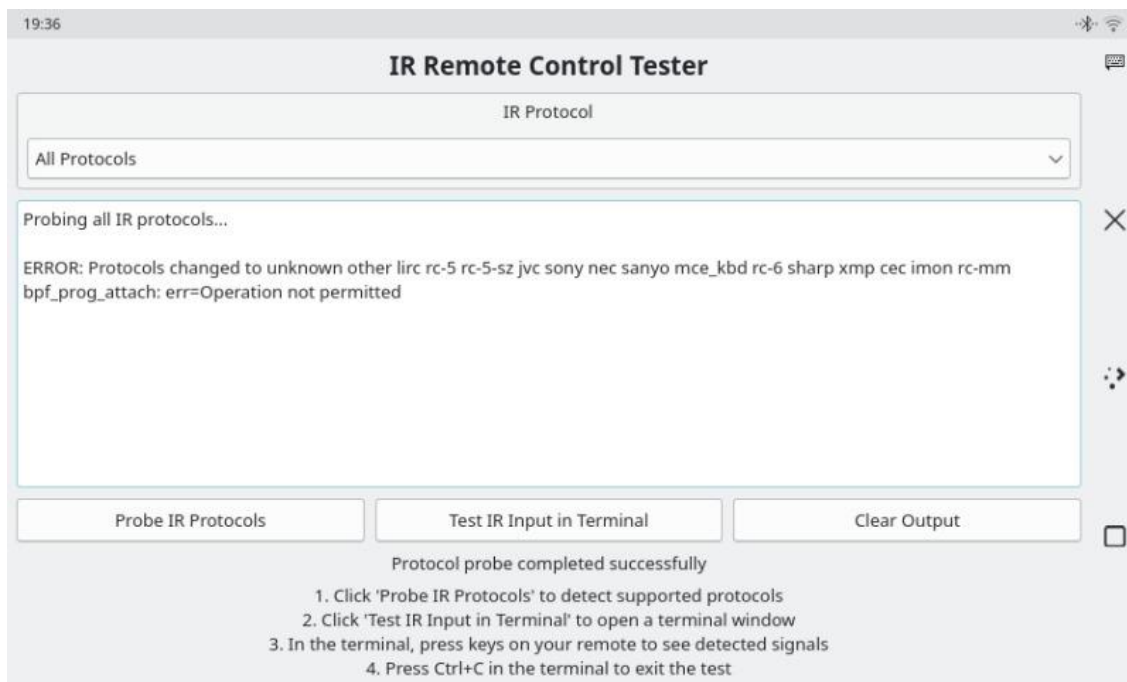


Fig. B.3. Changing All Protocols

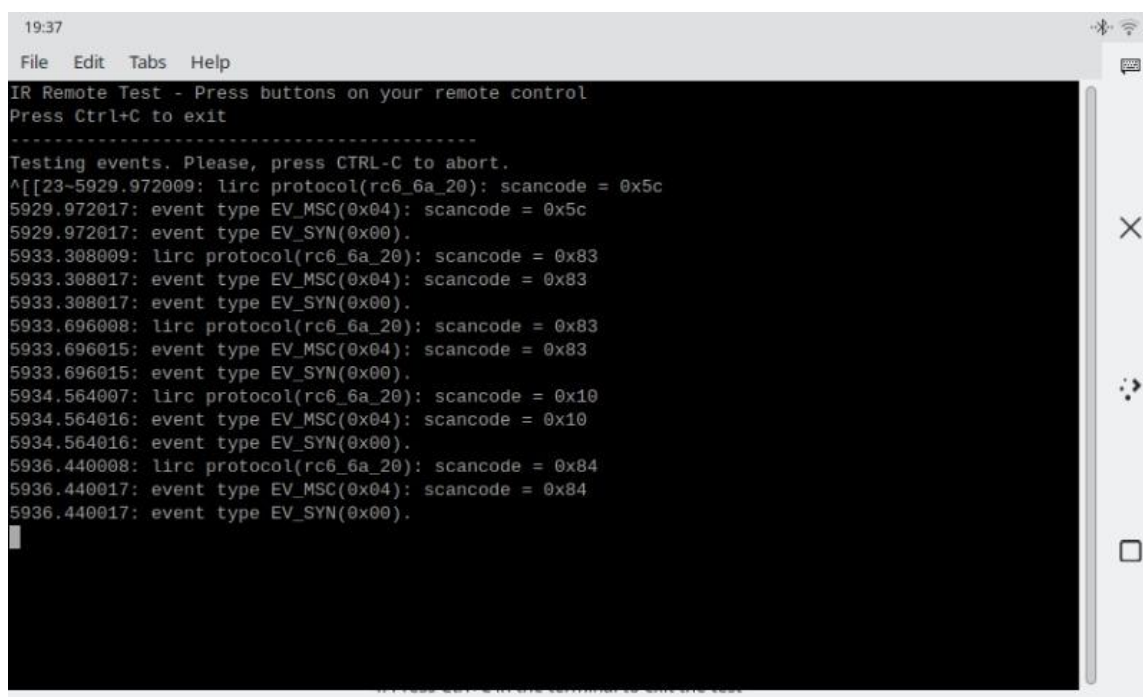


Fig. B.4. Terminal Running the Test



Fig. B.5. Applications List View

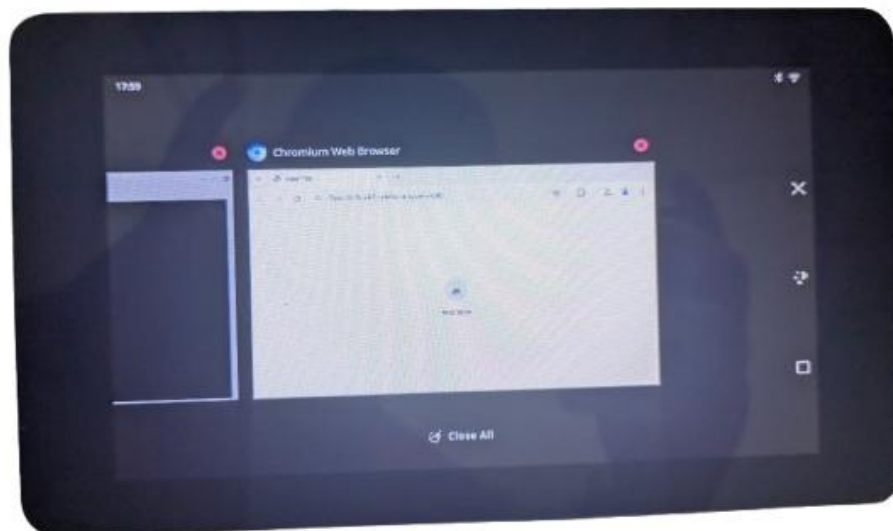


Fig. B.6. Task Switcher

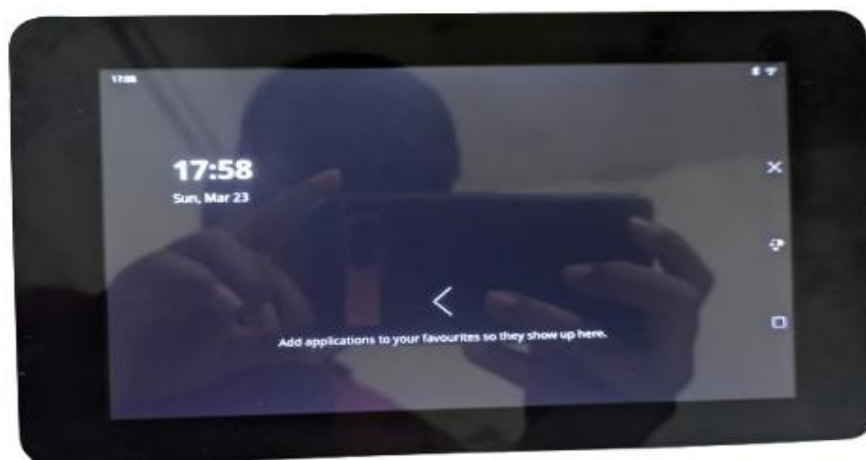


Fig. B.7. Home Screen



Fig. B.8. Customized KDE Control Centre



Fig. B.9. Custom GUI Settings



Fig. B.10. Lock Screen

C. PAPER PUBLICATION

2024 International Conference on Circuit Power and Computing Technologies (ICCPCT)

SMART HELMET USING IOT

Dr.P.Asha
Computer Science and Engineering
Sathyabama Institute of Science and
Technology
Chennai,India
ashapandian225@gmail.com

Akshara C V
Computer Science and Engineering
Sathyabama Institute of Science and
Technology
Chennai,India

Aakash S
Computer Science and Engineering
Sathyabama Institute of Science and
Technology
Chennai,India
aakashak@gmail.com

Acceptance – Intimation

Congratulations!!!

We are pleased to inform you that your paper has been accepted with major revisions for the for publication in “International Conference on Cognitive Computing and Artificial Intelligence” (ICCCAI-2025) held by Sathyabama Institute of Science and Technology.

“2nd International Conference on Cognitive Computing and Artificial Intelligence (ICCCAI-2025)” in association with National Kaohsiung University of Science and Technology (NKUST), Dambi Dollo University (DaDU), Lappeenranta-Lahti University of Technology (LUT) & Universiti Malaya (UM) will be held at Sathyabama Institute of Science and Technology (Deemed to be University), Chennai, India, during 27th - 28th March, 2025.

PAYMENT PROOF SCREENSHOT

Reference Number	HDFCN52025031824557830
From Account	50100173599720
Beneficiary Name	SATHYABAMA PUBLICATION
Beneficiary IFSC Code	IDIB000T020
Beneficiary Account Number / Credit Card Number	891734627
Bank Name	INDIAN BANK
Transfer Amount	9,500.00
Transfer Description	ICCCAI 495